Correct Optimizations: Phase 2 Report Based on "Minotaur: A SIMD-Oriented Synthesizing Superoptimizer" [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

1 Introduction

Minotaur [1] is a superoptimizer, that after computing optimizations, caches them in a database. For this purpose, it uses the original code segment as a key to find a rewrite/optimization, which is stored as the value of the key value pair. The intention behind the caching is to speedup the substantial runtimes of Minotaur for future compilations of the same or similar programs. Minotaur's compile times, with a cold cache, can easily be hundreds of times slower than with regular LLVM [1].

Currently, Minotaur uses its extracted LLVM IR code segments directly as cache-keys, storing them alongside rewrites, in Minotaur's own IR, as values. An example for a key-value pair is shown in Figure 2.

This approach can easily lead to avoidable cache misses because semantically equivalent code segments can have different keys. For example, if a cached segment has unused arguments while the current segment does not, the cache lookup fails and triggers a costly optimization search by Minotaur's solver.

To avoid this issue we attempt to increase the number of cache hits. This leads to reduced compile times of Minotaur even when starting from a cold cache. With the main challenge being to maintain the correctness of the cached optimizations.

2 Research Proposal

To address the issue of avoidable cache misses, our solution introduces an additional preprocessing step before a key is used for a cache lookup. It performs multiple code transformations to map semantically equivalent code to a "canonical" representation of the key.

The idea is that this leads to fewer cache entries, an increased cache hit rate and fewer slow optimization searches by Minotaur. For large enough projects this can already take effect during an initial build, even when starting with an empty cache, thereby reduce the compile time.

3 Canonicalization

To motivate the idea of canonicalization we start by defining the set of all valid LLVM IR snippets and call it the key space. The key space consists of equivalence classes of semantically equivalent keys, partitioning the key space. We call a set that contains one representative of each equivalence class a canonical key space. An ideal canonicalization transformation would map each input LLVM IR snippet to its representative in a canonical key space. Using such a transformation would enable reaching the optimal cache hit rate and avoid unnecessary expensive optimization searches by Minotaur, since there would be no cache misses anymore where an equivalent snippet has already been processed.

Finding such a transformation is hard, therefore we attempted to only shrink the size of each equivalence class. For this relaxation to be useful, each equivalence class has to be finite after the transformation is applied. This is because the probability that two randomly selected keys from the same equivalence class are identical depends on the size of that class. Because Minotaur only generates slices up to a fixed depth, it is possible to achieve finite equivalence classes with a transformation normalizing the names in a code snippet.

Our extension is placed as a pre- and postprocessing step around Minotaur's inference step. We intercept the LLVM IR produced by the slicer and perform transformations on it to produce a modified version of the input, which is then forwarded to the inference step. In the following this set of transformations is also called canonicalization. After the inference step is done and returns a rewrite in Minotaur's own IR, another set of transformations on the rewrite is performed. This process is called de-canonicalization from now on. Figure 1 gives an overview of how our extension integrates into Minotuar.

These transformation steps are always defined as a pair which we call a canonicalization-step. Each canonicalization-step in our implementation consists of a canonicalization function and a de-canonicalization function. The canonicalization-steps are ordered as to not interfere with each other. More

on that is discussed in section 3.2. During the canonicalization process every canonicalization function is applied iteratively in this predefined order. For de-canonicalization, the de-canonicalization functions are applied in the reverse order in which the canonicalization steps were applied. It is also possible to pass information between the pairs of canonicalization and de-canonicalization functions of each step. This enables the support for more complex transformations.

Due to the implementation of Minotaur and how we integrated our extension, all the de-canonicalization is performed by updating meta data, already required by minotaur for substitution of the rewrite, instead of relying on the explicit de-canonicalization function.

Our modified Minotaur version is available on GitHub [1].

3.1 Canonicalization Steps

In the following each canonicalization-step is described. They were selected such that each step maintains semantical equivalence, reduces the size of the key space and that it does not modify the code snippets depth. Here two code snippets are considered equivalent if they compute the same value and have the same effects on memory, including ordering of changes during execution, when using the same arguments. These three properties are shown for each canonicalization-step.

It might be possible to relax the correctness property to, for instance, allow the reordering of memory operations depending on the memory model. Similarly, the requirement of reducing the size of the key space could be relaxed such that it only holds over all steps but not for every individual step. Also, the depth constraint is not strictly required and could be dropped, but this would likely result in more complex solver queries which could drastically impact the overall runtime of Minotaur and offset any benefit gained from using canonicalization. This link between depth and runtime was already explored in the original paper [1].

An example of all the steps applied sequentially to an example input is shown in Figure 2

Eliminating Unused Function Arguments Some of the functions extracted by Minotaur have unused arguments which can be removed during canonicalization.

By removing those the number of functions with the same body but different signatures are reduced. Thereby decreasing the size of the key space.

Let $f : T_1 \times ... \times T_n \to R$ be an extracted function where argument t_i does not appear in the body of f. We can then define $g : T_1 \times ... \times T_{i-1} \times T_{i+1} \times ... \times T_n \to R$ where

$$\forall t_1 \in T_1, ..., t_n \in T_n : f(t_1, ..., t_n) = g(t_1, ..., t_{i-1}, t_{i+1}, ..., t_n)$$

This equality holds because t_i does not affect the computation in f's body. Setting the canonicalization $f' := g$, we obtain the same result for all valid inputs.

As shown in Figure 2 the rewrite does not have an explicit function signature. Therefore no de-canonicalization is required.

Modifying the function signature has no impact on the depth of the given code snippet.

Sorting of Function Arguments Another modification that is done to the function signature is that the function arguments are sorted. The arguments are sorted by their type-name and internally each set of arguments of the same type is sorted by when they are used inside the function body.

This defines a total order of function arguments for each function body and reduces the size of the key space.

Let $f : T_1 \times ... \times T_n \to R$ be an input function and f' the output function with reordered parameters according to permutation $\rho : [n] \to [n]$. Both functions accept the same set of arguments, just in different positions. For any input tuple $(t_1, ..., t_n)$, the function body computes the same result regardless of parameter order because the body operates on argument values, not their positional indices:

$$f(t_1, ..., t_n) = f'(t_{\rho^{-1}(1)}, ..., t_{\rho^{-1}(n)}) = \text{body}(t_1, ..., t_n)$$

Thus parameter reordering does not affect semantic equivalence.

Because Minotaur maintains a mapping between the extracted functions arguments and the original code the snippet was extracted from, we can simply update this mapping and do not require explicit de-canonicalization.

Since only the function signature is modified there is no change to the depth.

Renaming of Function Arguments After the previous two transformations the signature is almost unique for each function body with the exception of the naming. To resolve this each argument is renamed depending on its position in the signature.

Because now the names of all function arguments are defined by their position all functions with a given signature and body map to the same key.

Given a function $f(t_1, ..., t_n) := g(t_1, ..., t_n)$ as the input to the transformation step and a function $f'(x_1, ..., x_n) := g(x_1, ..., x_n)$ as the output. If we apply f and f' to the same set of arguments $v_i, i \in [n]$ we get

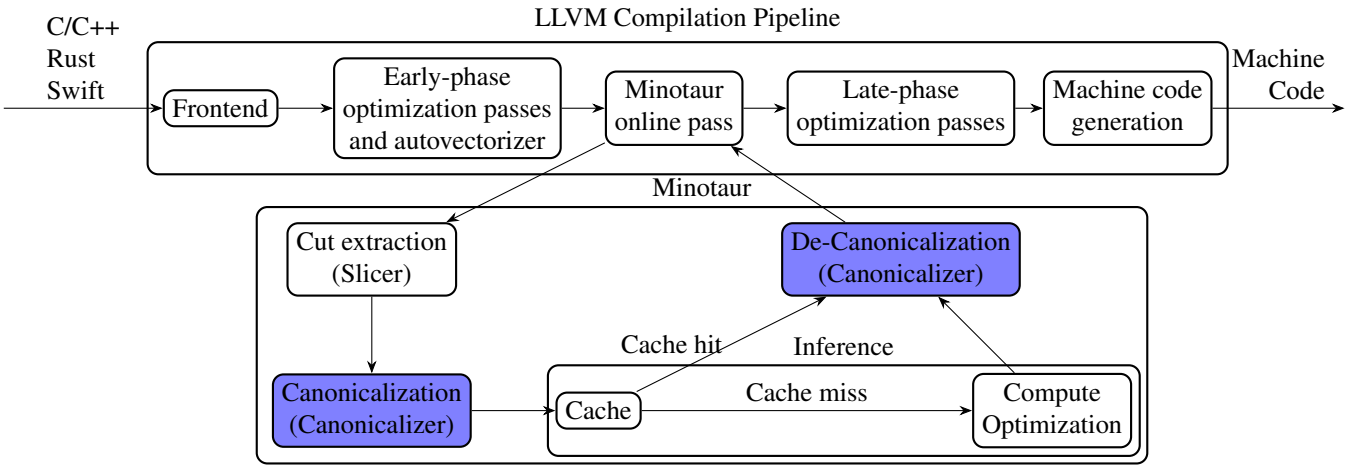
$$f(v_1, ..., v_n) = g(v_1, ..., v_n) = f'(v_1, ..., v_n)$$

This shows that renaming the arguments does not change the result of the function.

As mentioned previously, Minotaur maintains a mapping between the arguments of the extracted function and the

[1] Extended Minotaur Version

Figure 1: Structure of Minotaur and how the (De-)Canonicalization is integrated. The highlighted blocks were added.



original code snippet from which it was derived. This mapping does not rely on names but instead uses references to objects that possess names, eliminating the need for de-canonicalization.

Renaming does not impact the depth of the function.

Replacing Greater-Than with Less-Than Comparison operations often have multiple semantically equivalent ways of being expressed. To address this every greater-than or greater-equal comparison is replaced with a less-than or less-equal comparison respectively and the operands are swapped.

This reduces the size of the key space since now only less-than and less-equal comparisons are possible.

The correctness of the transformation follows from the "lemma" below.

$$\forall x \forall y.(x\ P\ y \iff y\ P'\ x)$$
$$with(P,P') \in (>,<),(\geq,\leq)$$

For this transformation no de-canonicalization is needed.

The depth is not impacted, since only the predicate of a comparison operator is modified.

Non-strict Operator Replacement This transformation replaces non-strict comparison operators with strict ones whenever possible, subject to certain constraints.

The replacement is applied only when the comparison involves constant operands, ensuring that the depth remains unchanged and that no additional operations are introduced.

By reducing the number of distinct ways to express equivalent comparisons, this transformation shrinks the key space.

Each value of LLVM's built-in integral and floating-point types represents a specific real number, with the exception of the special values *NaN* and $\pm$inf. The real numbers form a total order; consequently, the set of all non-special values of each type also forms a total order. For floating-point types, this order can be extended by defining $-$inf as the minimum element and $+$inf as the maximum element. Within a total order each value, with the exception of the minimum and maximum, have a greater and smaller neighbor.

In a total order, every value except for the minimum and maximum has both an immediate predecessor and an immediate successor. The minimum only has a successor, and the maximum only has a predecessor. Due to this property, non-strict comparison operators can be replaced by strict ones by appropriately adjusting a constant operand, provided that a suitable neighboring value exists.

For example, given an expression of the form $x \leq c$ where x is a variable and c is a constant, the expression can be transformed into $x < c + \varepsilon(c)$ where $\varepsilon(c)$ is the smallest representable increment at c (e.g., 1 for integers). Formally:

$$\forall x.(x \leq c \iff x < c + \varepsilon(c))$$

Similar arguments apply for $\geq$ and for cases where the constant appears on the left-hand side of the comparison.

No de-canonicalization is required for this transformation.

3.2 Ordering of the Canonicalization Steps

It is important to order the canonicalization-steps such that they do not interfere with previous ones. In our case this is still relatively simple since we only have two constraints.

First, the argument renaming step should be run after the last modification to functions arguments. In our case this implies that it has to run after the elimination of unused arguments step and after the argument sorting step.

Second, the sorting of function arguments has to take place after the last modification to the function body in which the

Figure 2: Example applying canonicalization steps (changes highlighted).

```
input/key:
define i1 @cut(i32 %a, float %b, i32 %c,
               float %d) {
       %1 = fcmp uge float %b, %d
       %2 = icmp sle i32 %c, 15
       %3 = and i1 %1, %2
       %4 = and i1 %3, 0
       ret i1 %4
}
1. unused argument elimination:
define i1 @cut (float %b, i32 %c, float %d) {
       %1 = fcmp uge float %b, %d
       %2 = icmp sle i32 %c, 15
       %3 = and i1 %1, %2
       %4 = and i1 %3, 0
       ret i1 %4
}
2. replacing greater-than with less-than:
define i1 @cut(float %b, i32 %c, float %d) {
       %1 = fcmp ule float %d, %b
       %2 = icmp sle i32 %c, 15
       %3 = and i1 %1, %2
       %4 = and i1 %3, 0
       ret i1 %4
}
```

```
3. replacing non-strict operators:
define i1 @cut(float %b, i32 %c, float %d) {
       %1 = fcmp ule float %d, %b
       %2 = icmp slt i32 %c, 16
       %3 = and i1 %1, %2
       %4 = and i1 %3, 0
       ret i1 %4
}
4. sorting arguments:
define i1 @cut (float %d, float %b, i32 %c) {
       %1 = fcmp ule float %d, %b
       %2 = icmp slt i32 %c, 16
       %3 = and i1 %1, %2
       %4 = and i1 %3, 0
       ret i1 %4
}
5. renaming argument:
define i1 @cut (float %__canonArg, float %__canonArg1,
                i32 %__canonArg2) {
       %1 = fcmp ule float %__canonArg, %__canonArg1
       %2 = icmp slt i32 %__canonArg2, 16
       %3 = and i1 %1, %2
       %4 = and i1 %3, 0
       ret i1 %4
}
output/value:
(reservedconst i1 |i1 false|)
```

usage order of the arguments is changed. This means concretely that it has to be run after the step replacing greater than with less than operators.

The following order fulfills these constraints:

1. Elimination of unused Function Arguments
2. Replacing Greater-Than with Less-Than
3. Non-strict Operator Replacement
4. Sorting of Function Arguments
5. Renaming of Function Arguments

4 Evaluation

For the evaluation we used an AMD Ryzen 9 5900 CPU which has 12 cores running at up to 4.8 GHz. All benchmarks were built using the $-O3$ and $-march = native$ flags. We based our evaluation on the latest version of Minotaur [2] that did not include any new features but all the available bug fixes.

The benchmark was built once using Minotaur with canonicalization module disabled and once with canonicalization module enabled. Since our canonicalization steps are compositional we measured the effect each step and combination of steps have. Furthermore, we ran the provided test and benchmark suite.

[2] Updated Minotaur Version

We chose to benchmark libgmp-6.2.1 as the original paper did [1]. Due to generally long compile times of Minotaur, we decided against benchmarking additional code like libYUV, like the original paper did.

4.1 Results

Building libgmp using our augmented version of Minotaur showed a speedup of 1.31x compared to regular Minotaur. This can also be seen in the bottom graph of Figure 3, which is why the orange plots stop earlier. In this graph it can also be seen that the respective curves have a similar shapes, but are shifted on the time axis. Overall, our canonicalizations only added 5.68 seconds of total CPU time for the build of libgmp.

The overall cache size using our augmented version was only 78.3% of the cache size of regular Minotaur, with 17734 key value pairs. When looking at Figure 3 it can be seen that the overall number of queries was very similar between the two versions. On the other hand, the number of cache hits for our augmented version was 12.7% greater, giving a overall cache hit rate of 67.7%, while regular Minotaur only achieved 59.2%. This is also reflected in the 28.6% higher number of solver invocations by regular Minotaur. In cases where the solver takes to long to find a beneficial rewrite, it times out. Our extended version shows a slightly lower timeout rate,

relative to the total number of solver invocations, of 25.1% compared to 27.2% compared to regular Minotaur. When looking at cache hits where the cached key also has a solution associated with it, this case is denoted as *sol* in Figure 3, we can see that this increased by 50% for our approach. Similar improvements, as the once observed to overall speedup , were also measured in the total CPU time it took to build libgmp, where a speedup of 1.37x was measured.

As mentioned earlier we used a updated version of Minotaur including new bug fixes. The tests included by libgmp were run and passed both for regular Minotaur and for our extended version.

In addition to building libgmp using Minotaur we also build it using standard LLVM. When comparing the results of running the benchmarks between standard LLVM and Minotaur we got the same inconclusive results as in our last report i.e. that Minotaur does not improve nor degrade the performance on average. When comparing the results of running the benchmarks between regular Minotaur and our extended version there is almost no difference across all measured values.

5 Discussion

The speedup measured by our extension is caused by the increase in the cache hit rate which leads to fewer long running solver invocations, which was the main goal of our proposed solution. This was achieved even though only very simple transformations were applied. Additional, potentially more complex transformations could lead to even more gains.

The differing numbers of queries, which can be observed in Figure 3, are also present between two runs of regular Minotaur. This shows that this inconsistent behavior is not caused by the addition of our module. It is not clear if this is caused by LLVM producing different LLVM IR between runs or if it is caused by Minotaur.

The passing libgmp tests indicate that our canonicalization preserves the semantics of the keys, suggesting that the implementation is correct.

6 Conclusion

Overall, it can be said that canonicalization was able to increase the cache hit rate for Minotaur and to thereby lower the number of long running solver invocations. This directly lead to a speedup of 1.31x when compiling libgmp. Due to time constraints only libgmp was build, because building of libgmp took, using both the regular Minotaur version and our extended version, around 86 hours. Further benchmarks could be interesting to gain a more representative picture of the effectiveness of canonicalization.

To improve our approach, one could attempt to perform a more structured search for additional transformations. One way of doing this could be finding syntactically equivalent rewrites in the cache and comparing their respective keys, to derive new transformations.

References

[1] Zhengyang Liu, Stefan Mada, and John Regehr. “Minotaur: A SIMD-Oriented Synthesizing Superoptimizer”. In: *Proceedings of the ACM on Programming Languages (OOPSLA2)* 8.OOPSLA2 (Oct. 2024). https://doi.org/10.1145/3689766, pp. 1–25. DOI: 10.1145/3689766.

Figure 3: Shows the comparison of cache metrics collected during the compilation of libgmp by the unmodified version of Minotaur (baseline in blue) and the extended version of Minotaur (canon in orange). The top row shows the total number of queries (queries), cache hits (hits), calls to the solver after a cache miss (solver_calls), solver timeouts (timeouts), rewrites successfully queried from the cache (sols) and the time spend in the solver (total_solver_time) over all build units.